The Economics of Technical Debt in Agentic Software Engineering

bordumb · bordumbb@gmail.com

4 July 2026 · draft v0 — **all results are synthetic placeholders**

Abstract

Autonomous coding agents change the economics of technical debt in one decisive way: they collapse the cost of *issuing* debt without changing the cost of *servicing* it. Generation now scales with compute while the two traditional brakes on defect accumulation — human review and honest self-assessment — do not: review capacity is fixed, and an agent's completion report is least reliable exactly when the work is wrong. We develop an economic model of this regime. Work items completed by an agent carry a **false-done rate** φ: the probability that work declared complete fails an oracle the agent never saw. Undiscovered false-dones accrue *interest* — every dependent change built on a latent defect raises its eventual repair cost — and we derive a solvency condition under which expected repair cost stays finite, showing that agentic build-on rates push systems toward the divergent regime. We then price the remedy. A **verification gate** — an independently executed, falsification-tested acceptance layer standing between the agent and the "done" signal — is modeled as a *costly state verification* contract: it pays a fixed, measurable premium in the cheapest resource in the loop (tokens) to avoid a heavy-tailed loss denominated in the most expensive (engineer attention and compounding rework). This yields a participation inequality whose terms are all measurable. Finally, we report a pre-registered two-model, two-arm evaluation on an externally-authored benchmark with held-out oracles (148 tasks; a small and a mid-tier model; 0% vs 100% gating at matched token budgets). In the placeholder results, gating cuts shipped-bad-work by 16–19 points, the token premium is 3–4×, and the break-even downstream cost of a single false-done is under one minute of engineer time — implying a premium-to-expected-payout ratio near 1:100, and a substitution frontier on which a gated small model undercuts an ungated mid-tier model once attention costs are priced in.

⚠ Draft status — the data below is synthetic

Every empirical value in §5 and §6 of this draft — counts, rates, intervals, costs, and the debt projections — is a **synthetic placeholder**, not a measurement. The numbers are the study's *pre-registered predictions*, written down before any cell of the experiment has run, and typeset here so the paper's analytical pipeline is fixed in advance of the data. Tables carrying placeholder values are marked **(synthetic)**. When the registered runs complete, every marked value will be replaced by the measured one — including any that falsify the paper's thesis — and this notice will be removed. §2–§4 (the model, the mechanism, and the experimental design) contain no synthetic content.

1. Introduction

1.1 Debt at machine speed

Technical debt began as a lending metaphor: shipping expedient code is a loan against future rework, and the unpaid principal charges interest as every subsequent change costs more than it should (Cun-

ningham 1992). Later work made the metaphor load-bearing — debt has identifiable principal, recurring interest, a lifecycle, and a portfolio that can be managed or mismanaged (Kruchten, Nord, and Ozkaya 2012), and the empirical software-engineering literature established the underlying cost curve: a defect grows more expensive the later it is found (Boehm 1981). By 2022 the stock of unpaid principal was estimated at $1.52 trillion for the United States alone, inside $2.41 trillion of annual cost attributable to poor software quality (Krasner 2022).

Those estimates describe the *human* borrowing rate. Autonomous coding agents change the rate. Generation throughput now scales with compute spend, and the early field data suggests the borrowing has already begun: in the largest longitudinal analysis of AI-era code, duplicated blocks grew eightfold in a single year while refactoring collapsed from 24% of changed lines to under 10% — the first year on record in which copy-paste exceeded refactoring (Harding and Kloster 2025). Fleet-level delivery data agrees: a 25% increase in AI adoption is associated with an estimated 7.2% *decrease* in delivery stability (DORA / Google Cloud 2024). And the borrower's own ledger cannot be trusted: in a randomized controlled trial, experienced developers using AI assistance were 19% slower while believing themselves 20% faster (Becker et al. 2025) — a 39-point gap between perceived and actual outcomes, in the direction that issues more debt.

1.2 The unit of account is the "done" signal

We take as primitive not lines of code but the **completion claim**: the moment an agent (or a human) declares a work item done. Each such claim is either honest (the work satisfies its intent) or a **false-done** — and the false-done, not the defect, is the economically active object. A defect that is *known* is a priced liability: it enters a backlog, is scheduled, and compounds slowly. A defect wrapped in a "done" signal is an *unpriced* liability: it is believed, built upon, and discovered only after other work depends on it. The information asymmetry is classical — a market where quality is unobservable at purchase time is a market for lemons (Akerlof 1970) — with one aggravating novelty: the seller is an agent whose self-assessment demonstrably degrades exactly on the items that are wrong. Models cannot reliably referee their own work (Huang et al. 2024), optimizing against learned judges inflates reported quality while true quality falls (Choudhury 2025), and practitioner-grade verifiers accept large fractions of adversarial garbage (Ray 2026). In economic terms: the borrower misreports the loan, in good faith, at scale.

This paper makes four contributions:

1. **A model of debt issuance under autonomous generation** (§2): false-done rate, discovery hazard, build-on interest, a closed-form expected repair cost, and a *solvency condition* separating the regime where debt is financeable from the regime where expected interest diverges.
2. **The verification gate as an insurance contract** (§3): an independently-executed, falsification-tested acceptance layer (bordumb 2026) priced as costly state verification (Townsend 1979), with a participation inequality whose every term is measurable.
3. **A pre-registered measurement design** (§4): two models × two arms on an externally-authored benchmark with held-out oracles at matched budgets — and its results template (§5), populated in this draft with synthetic placeholder values equal to the pre-registered predictions.
4. **The pricing consequences** (§6–§7): break-even thresholds, the substitution frontier between gated-cheap and ungated-capable models, fleet-level debt projections, and why the *verified-done signal* — not model capability — is the natural contractible unit for autonomous work.

2. A model of debt issuance under autonomous generation

2.1 Primitives

An agent fleet completes work items at rate Λ (items/day). Each completed item is declared done. With probability φ — the **false-done rate** — the declaration is wrong: the item fails an oracle O representing its true intent. The false-done then lies latent. Two clocks run on it:

- **Discovery.** The defect is found at a random time T with hazard δ (per day); we take $T \sim \mathrm{Exp}(\delta)$, so the mean time-to-discovery is $1/\delta$. Discovery is driven by testing, incident, or audit — capacities that do *not* scale with Λ.
- **Build-on.** While latent, dependent changes accumulate on top of the defect at rate β (changes/day). Each dependent change raises the eventual repair cost by a factor $(1 + g)$: rework must now also re-verify or unwind what was built on the falsehood.

The repair cost at discovery is therefore

$$c_r(T) \;=\; c_{r0}\,(1+g)^{\beta T},$$

with c_{r0} the cost of fixing the defect the day it was born — the familiar phase-cost curve (Boehm 1981), with the "phase" axis replaced by accumulated dependents.

2.2 Expected interest and the solvency condition

Writing $\gamma = \ln(1+g)$, the expected repair cost of one false-done is

$$L \;=\; \mathbb{E}\big[c_{r0}e^{\gamma\beta T}\big] \;=\; c_{r0}\,\frac{\delta}{\delta-\beta\gamma} \qquad \text{provided } \delta > \beta\gamma.$$

Proposition 1 (solvency). *Expected repair cost per false-done is finite if and only if the discovery hazard exceeds the interest rate: $\delta > \beta\gamma$. Otherwise the expectation diverges: the defect is, in expectation, built upon faster than it can be found.*

The ratio $M = \delta/(\delta-\beta\gamma) \geq 1$ is the **interest multiplier** — how much dearer the average defect becomes by lying latent. The proposition's force is directional: autonomous generation raises β (agents build on each other's output within hours, not sprints) and raises Λ, while leaving δ — testing, review, incident response — where it was. Human-paced development sits comfortably in the solvent regime; agentic development *drifts toward the divergence boundary by construction*, and crosses it when dependent work accrues faster than δ/γ. Past that point no finite repair budget prices the tail.

2.3 The debt stock and the velocity drag

Let $B(t)$ be the stock of undiscovered-plus-unrepaired defects. With repair capacity r (defects/day, bounded by staffing):

$$\frac{dB}{dt} \;=\; \Lambda\varphi \;-\; \min\{r,\ \delta B\},$$

so whenever issuance $\Lambda\varphi$ exceeds repair capacity r, the stock grows linearly without bound. Debt is not merely a liability on the books; it taxes the fleet's own throughput — navigating around latent defects, re-litigating broken assumptions, triaging incidents. We model the drag as

$$\Lambda_{\mathrm{eff}}(t) \;=\; \frac{\Lambda}{1+\kappa B(t)},$$

with κ the per-defect friction. The system has the classic debt-spiral phenomenology: issuance grows the stock, the stock cuts effective throughput, and — because review and repair are funded out of the same attention budget — the discovery hazard δ falls just as it is needed most.

2.4 The attention constraint

The traditional gate on φ is human review. Review capacity R (hours/day) audits a fraction

$$a \;=\; \min\left\{1,\; \frac{R}{\Lambda\, c_a}\right\}$$

of completions (at c_a hours per audit), catching a fraction q of the false-dones it inspects. Effective issuance into the debt stock is $\Lambda\varphi(1 - aq)$. The term that matters is a: as Λ scales with compute, $a \to 0$ — *unaudited* false-dones become the norm, not the exception. Every path out of this bind must make verification a manufactured good with the same scaling law as generation. That is the engineering content of §3.

3. The gate as an insurance contract

3.1 Mechanism

We evaluate the verification discipline of the companion framework (bordumb 2026), summarized in one paragraph. Every intended property of the work is a **claim** carrying an executable **probe** (GREEN / RED / BROKEN); a probe is admissible only after demonstrating it can fail — it must reject a known-bad **trap** (RED-first). A **gate** re-executes the probes independently and aggregates the verdict; the working agent cannot modify its own checks, and a deterministic controller — never the agent — decides doneness. "Done" is thus re-defined: not a self-report, but a gate verdict from checks that have *proven they can say no*.

Economically, the gate is **costly state verification** (Townsend 1979): the principal (whoever consumes the "done" signal) pays a verification cost to observe the true state of the work, rather than accepting the agent's report. Three parameters price it:

- ρ — the **price of trust**: the token/wall-clock multiplier of a gated run over a bare run;
- $\varphi_g < \varphi$ — the residual false-done rate behind the gate;
- η — the **refusal rate**: runs the gate ends red (budget exhausted, claim unproven). A refusal is not a failure of the mechanism; it converts a would-be unpriced liability into a priced one ("we do not have this yet"), which re-enters the queue at a known cost c_q.

3.2 The participation inequality

Let c be the bare per-item generation cost (tokens) and L the expected downstream loss per false-done from §2.2. Gating pays whenever

$$\underbrace{(\rho - 1)\,c}_{\text{premium}} \;+\; \underbrace{\eta\, c_q}_{\text{deductible}} \;<\; \underbrace{(\varphi - \varphi_g)\,L}_{\text{avoided loss}}. \tag{P}$$

Proposition 2 (premium bound). *At token prices p_t and attention prices p_h with p_h/p_t large, inequality (P) reduces to a threshold on the downstream loss alone: gating pays whenever a single false-done costs more than $L^* = \big[(\rho - 1)c + \eta c_q\big]/\Delta\varphi$ — a quantity denominated in tokens, to be compared against a loss denominated in engineer-hours and interest.*

The economics of the mechanism live in that denominational mismatch: the premium is paid in the *cheapest* resource in the loop and the avoided loss is paid in the *most expensive*, multiplied by the §2.2 interest term. §5 prices both sides with (placeholder) measurements.

3.3 Where the premium buys less: correlated authorship

The gate's checks are authored artifacts, and in the fully-autonomous configuration the *same model* writes the solution and its probe. A shared misreading of intent then produces a solution and a check that agree with each other and disagree with the oracle — the checks inherit the blind spots of the work (Huang et al. 2024; Helff et al. 2026). Formally, let m_a and m_p be the events that the actor and its probe miss a given defect; the gate's residual rate satisfies $\varphi_g = \varphi \cdot \Pr[\, m_p \mid m_a \,]$, and the **miss correlation** $\Pr[m_p \mid m_a] / \Pr[m_p]$ measures how much of the premium correlated authorship claws back. This is the quantity the falsification discipline (traps, fuzzing, cross-model adversaries) exists to push down (bordumb 2026; Ray 2026), and the paired analysis of §5.3 estimates it directly.

4. Experimental design (pre-registered)

The design measures every term of inequality (P) on externally-authored tasks. It is registered in the project ledger before any run; the analysis pipeline in this section is frozen, and §5's tables are its output format.

Substrate. BigCodeBench-Hard: 148 realistic library-usage tasks, each carrying a hidden `unittest` suite as a held-out oracle. The oracle is **quarantined**: it never enters any workspace; a separate evaluator applies it only after the agent process has exited (3 runs, majority verdict, flake rate reported).

Arms. A_0 (*bare*, 0% gating): the agent receives the task and an empty solution file, works as it pleases, and exits — a non-empty solution is a completion claim. A_3 (*gated*, 100%): the same workspace under the framework; the agent must author the claim, a RED-first probe derived from the task statement (it never sees the hidden suite), and at least one trap, then close under the gate. Gate-green is the completion claim; budget exhaustion with a red gate is a **refusal**.

Models. One small model (Claude Haiku 4.5) and one mid-tier model (Claude Sonnet 5), giving the first two points on the value-vs-capability curve and controlling for the confound that small models may fail at *operating* the harness rather than at the task — harness-operation failures are recorded separately from gate refusals.

Controls. Identical token cap per cell (60k) — budget-matched comparisons throughout, following the evaluation-rigor critique of (Hochlehnert et al. 2025); the same pinned task sample across all four cells (paired design); the cell matrix published before any run; McNemar's test on paired outcomes; Wilson 95% intervals on all rates; raw fractions always reported.

Primary quantities. Shipped-bad-work rate $\Pr[\text{declared} \wedge \text{oracle fails}]$ (the headline — robust to the arms' differing declaration rates), conditional false-done rate $\Pr[\text{oracle fails} \mid \text{declared}]$, price of trust ρ, refusal and process-failure rates, and the paired decomposition of §5.3.

5. Results — synthetic placeholders at the pre-registered values

> **Reading this section**
>
> The values below are the **pre-registered predictions** typeset through the frozen analysis pipeline — a dress rehearsal with stand-in numbers, marked **(synthetic)** throughout. Measured values replace them verbatim when the registered runs complete.

5.1 The headline cells

Cell	Declared	Refused (gate / process)	Oracle pass	Shipped bad (95% CI)	Conditional FDR
A_0 · Haiku	148/148	—	33 (22.3%)	**115/148 = 77.7%** [70.4, 83.7]	77.7%
A_3 · Haiku	124/148	24 (16 / 8)	37 (25.0%)	**87/148 = 58.8%** [50.7, 66.4]	70.2%
A_0 · Sonnet	148/148	—	57 (38.5%)	**91/148 = 61.5%** [53.5, 69.0]	61.5%
A_3 · Sonnet	133/148	15 (13 / 2)	66 (44.6%)	**67/148 = 45.3%** [37.5, 53.3]	50.4%

Table 1 — Per-cell outcomes on the pinned 148-task sample (synthetic).

Three (placeholder) findings. First, the bare small model ships bad work on **more than three of every four tasks** — completion claims from an ungated small agent carry almost no information. Second, gating removes **18.9 points** of shipped-bad from Haiku (McNemar on 38 discordant pairs, $\chi^2 = 20.6$, $p < 10^{-4}$) and **16.2 points** from Sonnet (44 discordant, $\chi^2 = 13.1$, $p \approx 3 \times 10^{-4}$) — at these two points the gate's absolute value is roughly flat in model capability, not an inverted U. Third, the gate does not tax outcomes: oracle pass rates *rise* slightly under gating in both models (+2.7 and +6.1 points), so the premium buys trust without costing solutions at matched budget.

5.2 The price of trust

Cell	Mean tokens/task	ρ	Token cost/task	Cost per oracle-pass
A_0 · Haiku	8.4k	1.0×	$0.018	$0.081
A_3 · Haiku	30.9k	**3.7×**	$0.065	$0.260
A_0 · Sonnet	7.6k	1.0×	$0.048	$0.125
A_3 · Sonnet	23.9k	**3.1×**	$0.152	$0.341

Table 2 — Token economics at list prices (synthetic).

On raw tokens the gate *loses*: cost per oracle-passing task is 2–3× higher gated. If tokens were the only cost, no rational buyer would gate. The next two subsections are why tokens are not the only cost.

5.3 Where the delta comes from: the paired decomposition

For each task the bare arm shipped bad, what did the gated arm do?

Of A_0-shipped-bad tasks…	Haiku (115)	Sonnet (91)
A_3 **fixed** (declared, oracle passes)	12 (10.4%)	23 (25.3%)
A_3 **refused** (red gate — loss priced, not shipped)	21 (18.3%)	11 (12.1%)
A_3 **also shipped bad** (shared blind spot)	82 (71.3%)	57 (62.6%)

Table 3 — Paired decomposition of the gate's effect (synthetic).

The composition differs by capability: the small model's delta is **refusal-driven** (it cannot repair what its probe catches, but the gate at least stops the shipment), the mid-tier model's is **repair-driven**. And the dominant bucket in both columns is the correlated-authorship residue of §3.3: in ~63–71% of bare failures, the self-authored probe shared the misreading and the gate honestly certified a wrong answer. The (synthetic) miss correlation is $\Pr[m_p \mid m_a]/\Pr[m_p] \approx 2.4$ — self-authored checks miss *with* their author far more than independently. This is the paper's measured argument for cross-model adversaries and differential probes (bordumb 2026), and the empirical shape matches the verifier-gaming literature (Ray 2026; Helff et al. 2026).

5.4 Attention-inclusive cost — the table that decides

Price a false-done at a deliberately conservative $L = \$45$ (45 minutes of blended engineer time at \$60/hour to detect, triage, and repair — *no* build-on interest), and a refusal at one automated retry:

Cell	Tokens	+ Expected downstream $\varphi \cdot L$	Total/task
$A_0 \cdot$ Haiku	\$0.018	\$34.97	**\$34.99**
$A_3 \cdot$ Haiku	\$0.065	\$26.46	**\$26.53**
$A_0 \cdot$ Sonnet	\$0.048	\$27.68	**\$27.73**
$A_3 \cdot$ Sonnet	\$0.152	\$20.39	**\$20.54**

Table 4 — Attention-inclusive cost per task (synthetic). Token spend is under 0.8% of total cost in every cell; the entire economics is the φL term.

Two consequences. **Break-even (Proposition 2):** at these numbers the gate pays whenever a single false-done costs more than $L^* = \$0.25$ for Haiku and \$0.64 for Sonnet — roughly **15 and 40 seconds of engineer time** respectively. Against the conservative $L = \$45$, the premium-to-avoided-loss ratio is approximately **1:34** (Sonnet) to **1:170** (Haiku): as insurance contracts go, the premium is negligible against the actuarial payout. **The substitution frontier:** the gated small model (\$26.53 all-in, 58.8% shipped-bad) strictly dominates the *bare mid-tier* model (\$27.73, 61.5%) — cheaper *and* cleaner once attention is priced — even though it loses on every token-only metric. Capability buys quality; verification buys *knowing which quality you got*; and at market prices the second is the better trade at the margin.

5.5 Fleet projection: debt at 90 days

Apply §2 with fleet parameters (10 agents × 30 tasks/day; discovery hazard $\delta = 1/14$ day^{-1}; build-on $\beta = 0.4$ dependents/day; $g = 6\%$ per dependent; repair capacity 120 defects/day):

	Bare fleet (Sonnet A_0)	Gated fleet (Sonnet A_3)
Defect issuance /day	184.5	135.9
Interest multiplier M (§2.2)	1.48×	1.48×
Undiscovered stock, day 90	≈ 5,800	≈ 1,430
Effective throughput, day 90 ($\varkappa = 5 \times 10^{-5}$)	−22%	−7%
Token premium /quarter	—	≈ $2.8k
Avoided expected repair /quarter	—	≈ $290k

Table 5 — 90-day fleet projection under the §2 model (synthetic).

The bare fleet loses between a fifth and a quarter of its own throughput to its accumulated debt within one quarter — the velocity drag of §2.3 made concrete — while the gated fleet's premium remains three orders of magnitude below its avoided expected repair. The solvency condition adds the cliff behind the slope: at agentic build-on rates ($\beta \gtrsim 1.2$/day at these parameters), $\beta\gamma$ crosses δ and the expected interest *diverges* — no repair budget suffices, and the only levers that restore solvency are raising δ (find defects faster) or cutting φ at issuance. The gate is the second lever; it is also, through its ledger, an instrument for the first.

6. Discussion: pricing autonomous work

The contractible unit. A market cannot contract on what it cannot verify (Akerlof 1970; Townsend 1979). Bare agent work offers only a self-report, so today's de-facto contract prices *capability* (tokens of a better model) as a proxy for quality — a lemons equilibrium in which every completion must be re-reviewed by the buyer or trusted on reputation. A gated completion changes the contractible unit: a **verified-done** — a completion accompanied by re-executable evidence and a *measured* residual false-done rate φ_g — is exactly the object a service-level agreement can bind. "95% of shipped work passes its held-out acceptance; refusals within budget b; evidence re-runnable by the buyer" is enforceable; "the model is smart" is not. We expect procurement of autonomous engineering to migrate to the verified unit for the same reason insurance markets demand audited books.

Moral hazard, resolved structurally. The agent grading its own work is a moral hazard problem no incentive scheme fixes — the agent is not strategic, merely miscalibrated, and §5.3's (placeholder) miss-correlation shows good faith does not help. The gate resolves it structurally rather than motivationally: the referee is code the actor cannot touch, the referee's competence is itself falsification-tested, and the residual hazard is a *published number* rather than an assumption.

What the premium buys as Λ grows. The attention constraint (§2.4) says human review vanishes as a fraction of completions; the gate's premium is the only term in (P) that scales *with* generation, because it is paid in the same currency. Verification-as-tokens is, to our knowledge, the only audit mechanism whose capacity curve matches the thing it audits.

Limitations. All results in this draft are synthetic placeholders (see notice). Beyond that: two models and one benchmark bound generality; the benchmark predates both models' training cutoffs, so contamination inflates both arms equally (the paired design protects the *delta*, not the levels); the oracle-quarantined design cannot rule out memorized solutions, only memorized *checks*; L and the §5.5 fleet parameters are assumptions to be replaced by calibration against incident data; and inequality (P) prices only defect debt, not the architectural erosion documented at the ecosystem scale (Harding and Kloster 2025), which plausibly makes our interest term an underestimate.

7. Related work

The debt metaphor and its formalization are due to (Cunningham 1992; Kruchten, Nord, and Ozkaya 2012), the defect phase-cost curve to (Boehm 1981), and the macro accounting to (Krasner 2022). Field evidence on AI-era code quality — clone growth, refactoring collapse, churn (Harding and Kloster 2025), throughput/stability trade-offs (DORA / Google Cloud 2024), and the perception gap in an RCT (Becker et al. 2025) — motivates our issuance model but measures neither false-done rates nor gate deltas. On the economics side we import adverse selection under unobservable quality (Akerlof 1970) and costly state verification (Townsend 1979); our contribution is to instantiate both with a mechanism whose verification cost is measurable in tokens. On the AI side, the failure of self-assessment (Huang et al. 2024), judge over-optimization (Choudhury 2025), verifier gaming (Helff et al. 2026; Ray 2026), the practitioner retreat to rule-based verifiable rewards (DeepSeek-AI 2025), and budget-matched evaluation rigor (Hochlehnert et al. 2025) jointly define the measurement standards this design follows. The mechanism under evaluation is the companion framework (bordumb 2026); this paper prices it.

8. Conclusion

Agentic software engineering industrializes the *issuance* of technical debt while leaving its *service* costs on a human clock — a mismatch that, by Proposition 1, is not merely expensive but eventually insolvent as build-on rates cross the discovery hazard. The way out is not better self-reports; it is making the "done" signal a manufactured, audited good. Priced as insurance, verification is anomalously cheap: the premium is paid in tokens — the one resource whose supply curve matches generation — and the avoided loss is paid in compounding engineer attention. In the pre-registered design of §4, every term of that trade is measurable, and in this draft's placeholder numbers the trade clears by one to two orders of magnitude, with a substitution frontier on which a gated small model undercuts an ungated capable one. The numbers will change when the registered runs complete — the notice at the top of this paper will not be removed until they do — but the accounting identity behind them will not: **in autonomous engineering, trust is the scarce factor of production, and it can now be bought at token prices.**

References

Akerlof, George A. 1970. "The Market for 'Lemons': Quality Uncertainty and the Market Mechanism." *The Quarterly Journal of Economics* 84 (3): 488–500.

Becker, Joel, Nate Rush, Elizabeth Barnes, and David Rein. 2025. "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." https://arxiv.org/abs/2507.09089.

Boehm, Barry W. 1981. *Software Engineering Economics*. Prentice-Hall.

bordumb. 2026. "Recurve: A Falsifiability-Gated Framework for Autonomous, Verifiable Problem-Solving." Companion paper, same repository (docs/papers/recurve-framework.md).

Choudhury, Sanjiban. 2025. “Process Reward Models for LLM Agents: Practical Framework and Directions.” https://arxiv.org/abs/2502.10325.

Cunningham, Ward. 1992. “The WyCash Portfolio Management System.” In *Addendum to the Proceedings of OOPSLA ’92 (Experience Report)*. Vancouver, BC.

DeepSeek-AI. 2025. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” https://arxiv.org/abs/2501.12948.

DORA / Google Cloud. 2024. “Accelerate State of DevOps Report 2024.” Google Cloud. https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report.

Harding, William, and Matous Kloster. 2025. “AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones.” GitClear research report. https://www.gitclear.com/ai_assistant_code_quality_2025_research.

Helff, Lukas, Quentin Delfosse, David Steinmann, Ruben Harle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. 2026. “LLMs Gaming Verifiers: RLVR Can Lead to Reward Hacking.” https://arxiv.org/abs/2604.15149.

Hochlehnert, Andreas et al. 2025. “A Sober Look at Progress in Language Model Reasoning.” In *Conference on Language Modeling (COLM)*. https://arxiv.org/abs/2504.07086.

Huang, Jie, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. “Large Language Models Cannot Self-Correct Reasoning Yet.” In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.01798.

Krasner, Herb. 2022. “The Cost of Poor Software Quality in the US: A 2022 Report.” Consortium for Information & Software Quality (CISQ). https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2022-report/.

Kruchten, Philippe, Robert L. Nord, and Ipek Ozkaya. 2012. “Technical Debt: From Metaphor to Theory and Practice.” *IEEE Software* 29 (6): 18–21.

Ray, Jaideep. 2026. “Before the Model Learns the Bug: Fuzzing RLVR Verifiers.” https://arxiv.org/abs/2606.01066.

Townsend, Robert M. 1979. “Optimal Contracts and Competitive Markets with Costly State Verification.” *Journal of Economic Theory* 21 (2): 265–93.